Figure 10-15 shows the dependency relationship between StockTaker and two inventory classes. Any StockTaker methods that work with an inventory object will need to be typed with each class, for example:

```
class StockTaker
{
    public function doInventoryWork(WarehouseInventory|ShopInventory
    $inventory)
```

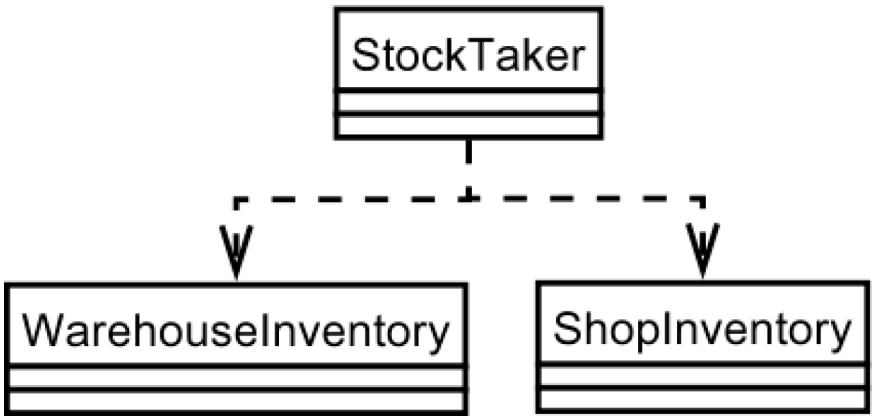


Figure 10-15. *StockTaker depends on multiple classes*

You might also remember that the UML diagram (Figure 10-11 in the previous section) showed an interface, Inventory, which was implemented by WarehouseInventory and ShopInventory, and StockTaker was instead made dependent on the interface. By making StockTaker dependent on an abstraction (for the purposes of this principle, an abstract base class and an interface are equivalent), we can simplify method declarations to use a single interface, for example:

```
class StockTaker
{
    public function doInventoryWork(Inventory $inventory)
```

If the problem domain expands and our software finds itself needing, say, KioskInventory, there's no need to modify the StockTaker class (provided that KioskInventory implements the Inventory interface).